

Stacks & queues

COMPLETE LEARNING GUIDE

What You'll Learn

Stacks and queues are the two simplest yet most broadly applicable abstract data types in DSA — a stack's Last-In-First-Out (LIFO) discipline underlies function call stacks, undo functionality, and expression parsing, while a queue's First-In-First-Out (FIFO) discipline underlies task scheduling and breadth-first traversal. This guide covers both structures' array and linked-list implementations, Java's idiomatic Deque-based usage for both (rather than the legacy, semi-deprecated Stack class), the classic balanced-parentheses and expression-evaluation applications, the monotonic stack technique for a whole family of "next greater/smaller element" problems, circular queues, and a first look at the priority queue (heap) as a specialized queue variant. These structures reappear constantly as building blocks inside larger algorithms — BFS uses a queue, DFS (iterative) uses a stack — so fluency here pays off throughout the rest of the DSA curriculum.

1. Stack: LIFO and Its Implementations

A stack supports push (add to the top), pop (remove and return the top), and peek (view the top without removing) — all in $O(1)$, whether backed by a dynamic array (pushing/popping at the end, which ArrayList already supports efficiently) or a singly linked list (pushing/popping at the head). Java's legacy Stack class extends Vector and is synchronized (with unnecessary locking overhead for single-threaded use) — the idiomatic modern choice is Deque<T> (specifically ArrayDeque), used as a stack via push()/pop()/peek(), which is unsynchronized and faster for typical single-threaded algorithmic code.

JAVA

```
Deque<Integer> stack = new ArrayDeque<>();    // idiomatic Java stack -- NOT
java.util.Stack
stack.push(1);
stack.push(2);
stack.push(3);
System.out.println(stack.pop());             // 3 -- last in, first out
System.out.println(stack.peek());            // 2 -- top, without removing
```

2. Queue: FIFO and Its Implementations

A queue supports enqueue (add to the back) and dequeue (remove from the front), both $O(1)$ when properly implemented — an array-backed queue needs to be circular (wrapping the front/back indices) to achieve $O(1)$ dequeue, since naively shifting every remaining element forward after removing from the front would cost $O(n)$. Java's Queue interface, implemented by LinkedList or ArrayDeque, provides offer() (enqueue) and poll() (dequeue)

with $O(1)$ amortized performance — `ArrayDeque` is generally preferred over `LinkedList` here too, for the same cache-locality reasons it's preferred as a `List` implementation.

JAVA

```
Queue<Integer> queue = new ArrayDeque<>(); // idiomatic Java queue
queue.offer(1);
queue.offer(2);
queue.offer(3);
System.out.println(queue.poll()); // 1 -- first in, first out
System.out.println(queue.peek()); // 2 -- front, without removing
```

3. Application: Balanced Parentheses

Checking whether a string's brackets are balanced and correctly nested is the canonical stack application: scan left to right, push every opening bracket onto the stack, and on every closing bracket, pop the stack and confirm it matches — a mismatch, or an attempt to pop an empty stack, immediately proves the string is unbalanced, and a non-empty stack at the end (unclosed openers) also proves it's unbalanced. This works because a stack's LIFO order exactly mirrors the nesting rule that the most recently opened bracket must be the next one closed.

JAVA

```
// Balanced parentheses check --  $O(n)$  time,  $O(n)$  space (worst case, all openers)
public static boolean isBalanced(String s) {
    Deque<Character> stack = new ArrayDeque<>();
    Map<Character, Character> pairs = Map.of(')', '(', ']', '[' , '}', '{');

    for (char c : s.toCharArray()) {
        if (c == '(' || c == '[' || c == '{') {
            stack.push(c);
        } else if (pairs.containsKey(c)) {
            if (stack.isEmpty() || stack.pop() != pairs.get(c)) {
                return false; // mismatched or nothing to match against
            }
        }
    }
    return stack.isEmpty(); // true only if every opener was matched
}
```

4. The Monotonic Stack Technique

A monotonic stack maintains its elements in strictly increasing or decreasing order at all times, popping elements that violate that order as each new element is processed — this pattern solves the entire family of "next greater element" / "next smaller element" problems in $O(n)$ total time, versus an $O(n^2)$ brute force that checks every pair. Each array element is pushed onto the stack exactly once and popped at most once across the whole algorithm, which is why the total work is $O(n)$ despite the nested-looking while loop inside the for loop — this amortized-cost argument is worth stating explicitly, since it looks like $O(n^2)$ at first glance.

JAVA

```
// Next Greater Element for every array position -- O(n) time via monotonic stack
public static int[] nextGreaterElement(int[] arr) {
    int[] result = new int[arr.length];
    Arrays.fill(result, -1);
    Deque<Integer> stack = new ArrayDeque<>(); // stores INDICES, kept in
    decreasing value order

    for (int i = 0; i < arr.length; i++) {
        while (!stack.isEmpty() && arr[stack.peek()] < arr[i]) {
            result[stack.pop()] = arr[i]; // arr[i] is the "next greater" for
            this popped index
        }
        stack.push(i);
    }
    return result;
}
// Time: O(n) -- each index is pushed once and popped at most once (amortized)
// Space: O(n) -- worst case, the stack holds every index
```

5. Circular Queues

An array-backed queue that simply advances a front index on dequeue eventually "leaks" usable space at the front of the array even though logically empty slots exist — a circular queue fixes this by wrapping both front and rear indices around modulo the array's capacity, reusing freed space instead of ever needing to shift elements or grow the array unnecessarily. Correctly distinguishing a full circular queue from an empty one (both can leave `front == rear`) typically requires either tracking a separate size counter or deliberately leaving one array slot always unused as a sentinel — a detail worth knowing if asked to implement one from scratch rather than relying on Java's `ArrayDeque`.

6. Priority Queue: A Specialized Queue Ordered by Priority

A priority queue dequeues not by arrival order but by priority — the smallest (or largest, with a custom `Comparator`) element is always removed first, regardless of insertion order — and Java's `PriorityQueue` is backed internally by a binary heap, giving $O(\log n)$ for `offer()` and `poll()`, and $O(1)$ for `peek()`. This is a specialized enough and important enough structure that it's covered in full depth alongside heaps in the DSA Advanced module's `Trees & Graphs` and `Sorting` submodules, but it's worth introducing here as conceptually "a queue where priority replaces arrival order" — the same enqueue/dequeue mental model, with a different ordering rule underneath.

JAVA

```
PriorityQueue<Integer> minHeap = new PriorityQueue<>(); // smallest element
minHeap.offer(5);
minHeap.offer(1);
minHeap.offer(3);
System.out.println(minHeap.poll()); // 1 -- smallest, not first-inserted
// PriorityQueue<Integer> maxHeap = new
// PriorityQueue<>(Comparator.reverseOrder());
```

Common Pitfalls

- Using `java.util.Stack` instead of `Deque/ArrayDeque` — `Stack` is a legacy synchronized class with unnecessary locking overhead for typical single-threaded algorithmic code.
- Implementing a queue with a plain array and naively shifting elements after every dequeue, turning an $O(1)$ operation into $O(n)$ — a circular buffer avoids this.
- Forgetting to check for an empty stack before popping in a balanced-parentheses check, throwing a `NoSuchElementException` instead of correctly returning "unbalanced."
- Believing the nested while loop inside a monotonic stack algorithm makes it $O(n^2)$ — each element is pushed and popped at most once total, making it $O(n)$ amortized.
- Confusing a full vs. empty circular queue when both can produce `front == rear` — forgetting to track size explicitly or reserve a sentinel slot.
- Assuming `PriorityQueue.poll()` returns elements in insertion order — it returns the smallest (or comparator-defined highest-priority) element regardless of when it was added.
- Using a plain `Queue` when what's actually needed is priority-based ordering, missing that `PriorityQueue` exists specifically for that case.

Key Takeaways

- Stack: LIFO, push/pop/peek all $O(1)$; Queue: FIFO, enqueue/dequeue $O(1)$ with a properly circular or linked implementation.
- Use `Deque/ArrayDeque` for both stacks and queues in Java — not the legacy synchronized `Stack` class.
- Balanced-parentheses checking is the canonical stack application — LIFO order exactly mirrors bracket nesting rules.
- A monotonic stack solves "next greater/smaller element" problems in $O(n)$ amortized time, since each element is pushed and popped at most once.
- A circular queue reuses freed array space via modulo-wrapped indices instead of shifting elements or growing unnecessarily.
- `PriorityQueue` (a binary heap) dequeues by priority, not arrival order — $O(\log n)$ insert/remove, $O(1)$ peek.

Learn More

GeeksforGeeks — Stack Data Structure

<https://www.geeksforgeeks.org/dsa/stack-data-structure/>

GeeksforGeeks — Queue Data Structure

<https://www.geeksforgeeks.org/dsa/queue-data-structure/>

GeeksforGeeks — Next Greater Element (Monotonic Stack)

<https://www.geeksforgeeks.org/dsa/next-greater-element/>